DOCUMENT 4

DATABASE SCHEMA & TABLE DESIGN

Data Modeling in ServiceNow

Data Modeling in ServiceNow is a fundamental process of creating a structured framework for information. For ABC Corporation's ticket automation project, a robust database is required to accurately store support ticket details such as Issue type, Priority, and Assignment Group. This database serves as the essential 'Input' for the Flow Designer. Without a well-defined data model, automation triggers will not function correctly.

Custom Table vs Out-of-the-box (OOB) Tables

ServiceNow provides two primary types of tables:

- Out-of-the-box (OOB): Ready-to-use tables like Incident, Change, and Problem.
- Custom Tables: Tables created manually to meet specific business requirements.
- Decision: For ABC Corporation's specific requirements, creating a Custom Table named 'Operations related' is the best approach. This ensures the project logic remains isolated without affecting other standard system functions.

Creating Table - 'Operations related' (Part 1)

Steps to create a new table:

1. Filter Navigator: Type "Tables" and navigate to System Definition >> Tables.
2. New Button: Click on the 'New' button at the top.
3. Label: Enter Operations related.
4. Name: Once the label is entered, the system automatically generates the technical name u_operations_related (the 'u' prefix denotes a user-defined table).

Creating Table - 'Operations related' (Part 2)

Basic table configuration settings:

- Create Module: Checking this box creates a dedicated navigation shortcut for the table in the filter navigator.
- Mobile Module: Enables this table to be viewed and managed via the ServiceNow mobile application.
- Extends Table: Keep this as a standalone table (None) to ensure it operates independently within the project scope.

Table Properties - Label, Name, and Application Scope

- Label: The display name visible to users on the User Interface (e.g., Operations related).
- Name: The technical internal database name (e.g., u_operations_related).
- Application Scope: Set this to 'Global'. This allows the table and its data to be accessed across all applications, including Flow Designer and Reporting modules.

Field Configuration - 'Assigned to group' (Reference Field)

Configuring the group assignment column:

- Type: Reference.
- Reference Table: sys_user_group (This is the core ServiceNow table that stores all support groups).
- Purpose: This field is the key driver for automation; the Flow Designer uses this field to determine which support team should receive the ticket.

Field Configuration - 'Issue' (Choice Field)

Creating the issue categorization column:

- Type: Choice.
- Purpose: This helps identify the specific nature of the ticket. Using a dropdown (choice) ensures data consistency by preventing users from typing incorrect values.
- Mandatory: Set this field to 'True' so that a user cannot submit a ticket without selecting an issue type.

Choice List Management - Adding '404 Error' & 'Login issues'

Populating choices for the Issue field:

1. Choice List: Right-click the 'Issue' field and select 'Show Choice List'.
2. New Choices:
    - Unable to login to platform
    - 404 Error
    - Regarding User expired
    - Regarding Certificates

3. Values: Assign unique technical values and labels for each choice to ensure accurate processing by the automation engine.

Field Configuration - 'Priority' and 'Service request No'

- Priority: Type: Choice (Options: Critical, High, Moderate, Low). This defines the urgency of the support request.
- Service request No: Type: String. This serves as the unique identification number (ID) for every ticket raised in the system.

Auto-numbering and Prefix Configuration

Steps to auto-generate the 'Service request No':

1. Number Maintenance: Check the 'Auto-number' box in the table record.
2. Prefix: Set to SR (standing for Service Request).
3. Number of Digits: 4 digits (e.g., SR0001, SR0002).
4. Benefit: Automated numbering makes it much easier to track, search, and generate reports for support tickets.

UI Policy & Data Policy Overview

- UI Policy: Creates client-side rules such as "If Priority is 'Critical', then make the Description field mandatory." This improves the user experience and data quality.
- Data Policy: Protects the database from invalid data entries. Unlike UI policies, Data Policies are enforced at the server level, monitoring data coming through the UI as well as APIs.

Database Indexing & Performance

- Indexing: Applying indexes to 'Service request No' and 'Assigned to group' fields ensures that data searches and routing triggers execute with high speed.
- Performance Best Practices:
    1. Avoid creating unnecessary columns to keep the table lean.
    2. Extensive use of Reference fields is encouraged as it maintains relational integrity without degrading performance.
- Conclusion: This carefully designed table structure forms the foundation of the successful ticket automation system for ABC Corporation.